

DevOps Training Program

Hands-On Lab Workbook

Linux · Jenkins · AWS EC2 · IAM

Week-Long Lab Series | 6 Labs

Students will submit screenshots & output evidence for each lab

Lab Schedule Overview

Day	Lab #	Topic	Level
Day 1 – Mon	Lab 1	Linux Essentials on EC2	Beginner
Day 2 – Tue	Lab 2	AWS EC2 Launch & Management	Beginner
Day 3 – Wed	Lab 3	AWS IAM Users, Roles & Policies	Intermediate
Day 4 – Thu	Lab 4	Jenkins Freestyle Job & Build Triggers	Intermediate
Day 5 – Fri	Lab 5	Jenkins Pipeline with GitHub	Intermediate
Day 6-7 – Weekend	Lab 6	End-to-End CI/CD: Jenkins + EC2 Deploy	Advanced

NOTE

Jenkins is already installed on your EC2 instance. Each lab builds on the previous one. Always save your screenshots as you go — you will need them for submission.

LAB 1

Linux Essentials on EC2



Duration: 2–3 Hours



Difficulty: Beginner ★

Lab Statement

In this lab, you will connect to your AWS EC2 Linux instance and practice essential Linux commands. You will navigate the file system, manage files and directories, handle user permissions, monitor system resources, and configure basic services — skills that form the backbone of all DevOps work.

Part A — Connecting to EC2 & Navigation

Connect via SSH and explore the file system:

```
ssh -i your-key.pem ec2-user@<your-ec2-public-ip>
pwd && whoami && uname -a
ls -la /      # List root filesystem
cd /var/log && ls -lh    # Explore logs
df -h && free -h    # Check disk and memory
```

Part B — File & Directory Management

```
mkdir -p ~/devops/labs/lab1
cd ~/devops/labs/lab1
echo 'Hello DevOps' > welcome.txt
cat welcome.txt
cp welcome.txt backup_welcome.txt
mv backup_welcome.txt archive/    # Create archive dir first
find /home -name '*.txt' 2>/dev/null
```

Part C — Permissions & User Management

```
chmod 755 welcome.txt && ls -l welcome.txt
sudo useradd devstudent && sudo passwd devstudent
sudo usermod -aG sudo devstudent
id devstudent
```

Part D — Process Monitoring & Services

```
ps aux | grep jenkins    # Check Jenkins process
top -b -n 1 | head -20   # System snapshot
sudo systemctl status jenkins
sudo systemctl status sshd
netstat -tlnp | grep 8080  # Verify Jenkins port
```

Expected Outcomes

✓	Successfully SSH into EC2 instance from local machine
✓	Create and manage directories/files using Linux commands
✓	Set correct file permissions using chmod and chown
✓	Add a new Linux user and assign sudo privileges
✓	View running processes and verify Jenkins is active on port 8080
✓	Understand the Linux filesystem hierarchy (/etc, /var, /home, /opt)

Student Submission Requirements

Screenshot 1	SSH terminal showing successful login (whoami + uname -a output)
Screenshot 2	File listing with permissions (ls -la) in your devops/labs/lab1 directory
Screenshot 3	Output of id devstudent showing sudo group membership
Screenshot 4	systemctl status jenkins showing active/running status
Text File	Paste the output of: df -h && free -h && netstat -tlnp grep 8080

LAB 2

AWS EC2 Launch & Instance Management



Duration: 2–3 Hours



Difficulty: Beginner ★

Lab Statement

In this lab, you will use the AWS Management Console to launch a new EC2 instance, configure Security Groups, connect to it, and practice instance lifecycle operations. You will also attach an EBS volume and understand EC2 pricing basics.

Part A — Launch an EC2 Instance

Log into AWS Console → EC2 → Launch Instance:

- Name: DevOps-Lab2-Instance
- AMI: Amazon Linux 2023 (Free Tier eligible)
- Instance Type: t2.micro
- Key Pair: Create new key pair → devops-key → Download .pem file
- Network: Default VPC, Enable Auto-assign Public IP
- Security Group: Create new → Allow SSH (22) from My IP, HTTP (80) from Anywhere

Launch and wait for Instance State: Running

Part B — Connect & Configure the Instance

```
chmod 400 devops-key.pem      # On local machine
ssh -i devops-key.pem ec2-user@<Public-IP>
sudo yum update -y
sudo yum install -y httpd git
sudo systemctl start httpd && sudo systemctl enable httpd
echo '<h1>DevOps Lab 2 - Working!</h1>' | sudo tee /var/www/html/index.html
```

Open a browser and go to <http://<your-ec2-public-ip>> — you should see the webpage.

Part C — EBS Volume Attachment

In EC2 Console → Elastic Block Store → Volumes:

- Create Volume → 8 GB, gp2, same AZ as your instance → Create
- Right-click the new volume → Attach Volume → Select your instance → /dev/xvdf

```
lsblk      # On EC2, verify new volume shows up
sudo mkfs.ext4 /dev/xvdf
sudo mkdir /data && sudo mount /dev/xvdf /data
df -h | grep /data    # Confirm mount
```

Part D — Instance Lifecycle

Practice Stop, Start, and Reboot from the EC2 Console. Note the public IP changes after Stop+Start (unless using Elastic IP).

Expected Outcomes

✓	Launch an EC2 instance with correct AMI, instance type, and key pair
✓	Configure Security Groups to allow SSH and HTTP traffic
✓	Connect to EC2 via SSH using the .pem key
✓	Install and run Apache web server on EC2
✓	Create and attach an EBS volume to the running instance
✓	Understand how instance Stop/Start affects public IP addresses

Student Submission Requirements

Screenshot 1	EC2 Console showing your instance in Running state with public IP visible
Screenshot 2	Browser showing your HTML page at <a href="http://<EC2-Public-IP>">http://<EC2-Public-IP>
Screenshot 3	SSH terminal showing lsblk and df -h output with the mounted EBS volume
Screenshot 4	EC2 Console Security Group rules showing SSH (22) and HTTP (80)
Note	Record what happened to the public IP after you stopped and restarted the instance

LAB 3

AWS IAM — Users, Roles & Policies



Duration: 2–3 Hours



Difficulty: Intermediate ★★

Lab Statement

Security is the #1 priority in AWS. In this lab, you will use the Principle of Least Privilege to configure IAM. You will create users, groups, custom policies, and a role that allows EC2 to access S3 — a very common real-world pattern used in DevOps pipelines.

Part A — Create IAM Users & Groups

AWS Console → IAM → User Groups → Create Group:

- Group Name: devops-team
- Attach Policy: AmazonEC2ReadOnlyAccess, AmazonS3ReadOnlyAccess
- Create Group

IAM → Users → Create User:

- Username: devops-student-01
- Enable: AWS Management Console access → Custom password
- Add to group: devops-team
- Download the .csv credentials file

Part B — Create a Custom IAM Policy

IAM → Policies → Create Policy → JSON tab. Paste this policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket", "s3:PutObject"],
      "Resource": ["arn:aws:s3:::devops-lab-bucket/*",
                  "arn:aws:s3:::devops-lab-bucket"]
    }
  ]
}
```

Policy Name: DevOpsS3Access → Create Policy

Part C — Create an IAM Role for EC2

This allows your EC2 instance to access S3 without storing credentials:

- IAM → Roles → Create Role → AWS Service → EC2 → Next
- Attach: DevOpsS3Access policy you just created

- Role Name: ec2-s3-access-role → Create Role

Attach the role to your EC2 instance:

- EC2 → Select your Lab 2 instance → Actions → Security → Modify IAM Role
- Select ec2-s3-access-role → Update

Part D — Test the IAM Role from EC2

SSH into your EC2 instance:

```
aws s3 ls      # Should list buckets (no credentials needed!)
aws sts get-caller-identity  # Shows the role being used
```

Create an S3 bucket and test:

```
aws s3 mb s3://devops-lab-$(date +%s)  # Create bucket
echo 'test' > test.txt && aws s3 cp test.txt s3://your-bucket-name/
aws s3 ls s3://your-bucket-name/
```

Expected Outcomes

✓	Create an IAM User Group with read-only EC2 and S3 permissions
✓	Create an IAM user with console access and assign to the group
✓	Write a custom IAM JSON policy with specific S3 permissions
✓	Create an IAM Role with EC2 as the trusted entity
✓	Attach the IAM Role to an EC2 instance (Instance Profile)
✓	Run AWS CLI commands from EC2 using the role (no Access Keys needed)

Student Submission Requirements

Screenshot 1	IAM Console showing devops-student-01 user inside devops-team group
Screenshot 2	IAM Policy JSON of your DevOpsS3Access custom policy
Screenshot 3	EC2 Console showing ec2-s3-access-role attached to your instance
Screenshot 4	SSH terminal showing output of: aws sts get-caller-identity
Screenshot 5	Terminal output of aws s3 ls and successful file upload to S3 bucket

LAB 4

Jenkins Freestyle Job & Build Triggers



Duration: 2–3 Hours



Difficulty: Intermediate ★★

Lab Statement

Jenkins is already installed on your EC2 instance. In this lab, you will access the Jenkins dashboard, create your first Freestyle job, configure different types of build triggers, use shell build steps, and understand Jenkins workspace, console output, and build history.

Part A — Access Jenkins & Initial Setup

Open your browser and navigate to: `http://<EC2-Public-IP>:8080`

```
# If locked out, get the initial admin password:  
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Complete the Jenkins setup wizard: Install Suggested Plugins → Create Admin User → Save & Continue.

Part B — Create Your First Freestyle Job

Jenkins Dashboard → New Item → Enter name: Lab4-HelloWorld → Freestyle project → OK

Configure the job:

- Description: My first Jenkins build job
- Build Steps → Add build step → Execute shell

```
#!/bin/bash  
echo '=====  
echo 'Hello from Jenkins Build!'  
echo 'Build Number: '$BUILD_NUMBER  
echo 'Build Date: '$(date)  
echo 'Hostname: '$(hostname)  
echo 'Jenkins Home: '$JENKINS_HOME  
echo '=====  
ls -la $WORKSPACE
```

Save → Build Now → Click the build number → Console Output

Part C — Configure Build Triggers

Edit the job → Build Triggers:

- Enable: Build periodically → Schedule: `H/5 * * * *` (every 5 minutes)
- This uses cron syntax: minute hour day month weekday

Also configure a Parameterized Build:

- General → This project is parameterized → Add Parameter → String Parameter
- Name: `STUDENT_NAME`, Default Value: `YourName`

```
echo 'Build run by: '$STUDENT_NAME
```

Trigger 3 manual builds and observe the build history. Note how BUILD_NUMBER increments.

Part D — Post-Build Actions & Artifacts

Add a build step to generate an output file:

```
echo 'Build '$BUILD_NUMBER' completed at '$(date) > build_report.txt  
cat build_report.txt
```

Post-build Actions → Archive the artifacts → Files to archive: build_report.txt → Save

Run a build and download the artifact from the build page.

Expected Outcomes

✓	Access Jenkins dashboard and complete initial configuration
✓	Create a Freestyle job with a multi-step shell build script
✓	Use Jenkins environment variables (BUILD_NUMBER, WORKSPACE, JENKINS_HOME)
✓	Configure cron-based build triggers (Build Periodically)
✓	Create and use parameterized builds
✓	Archive build artifacts and download them from the Jenkins UI

Student Submission Requirements

Screenshot 1	Jenkins dashboard showing your Lab4-HelloWorld job
Screenshot 2	Console Output of any successful build run
Screenshot 3	Build History showing at least 3 successful builds (green checkmarks)
Screenshot 4	Archived artifact (build_report.txt) visible on the build page
Screenshot 5	Build Triggers configuration showing your cron schedule

LAB 5

Jenkins Pipeline with GitHub Integration



Duration: 3–4 Hours



Difficulty: Intermediate ★★

Lab Statement

In this lab, you will migrate from Freestyle jobs to the modern Jenkins Pipeline using Groovy DSL (Jenkinsfile). You will connect Jenkins to a GitHub repository, define a multi-stage pipeline with Build, Test, and Deploy stages, and trigger automated builds on code push.

Part A — GitHub Repository Setup

Create a free GitHub account if you don't have one. Create a new public repository named devops-jenkins-lab.

On your EC2 instance, push a sample project:

```
sudo yum install -y git
git config --global user.name 'Your Name'
git config --global user.email 'your@email.com'
mkdir ~/jenkins-project && cd ~/jenkins-project
git init && git remote add origin https://github.com/YOUR_USERNAME/devops-jenkins-lab.git
```

Part B — Create the Jenkinsfile

Create a file named Jenkinsfile (no extension) in your project root:

```
cat > Jenkinsfile << 'EOF'
pipeline {
    agent any
    environment {
        APP_NAME = 'devops-app'
        VERSION   = "1.0.${BUILD_NUMBER}"
    }
    stages {
        stage('Checkout') {
            steps {
                echo 'Pulling source code...'
                checkout scm
            }
        }
        stage('Build') {
            steps {
                echo "Building ${APP_NAME} version ${VERSION}"
                sh 'mkdir -p build && echo built > build/artifact.txt'
            }
        }
    }
}
```

```

}
stage('Test') {
  steps {
    echo 'Running tests...'
    sh 'echo TEST PASSED >> build/artifact.txt'
  }
}
stage('Deploy') {
  steps {
    echo "Deploying version ${VERSION} to server"
    sh 'cp build/artifact.txt /tmp/deployed_app.txt'
    sh 'cat /tmp/deployed_app.txt'
  }
}
}
post {
  success { echo 'Pipeline completed successfully!' }
  failure { echo 'Pipeline FAILED. Check logs above.' }
}
}
EOF
git add Jenkinsfile && git commit -m 'Add Jenkinsfile'
git push -u origin main

```

Part C — Create Pipeline Job in Jenkins

Jenkins → New Item → Pipeline → Name: Lab5-Pipeline → OK

- Pipeline Definition: Pipeline script from SCM
- SCM: Git → Repository URL: https://github.com/YOUR_USERNAME/devops-jenkins-lab.git
- Branch Specifier: */main → Script Path: Jenkinsfile

Save → Build Now. Observe the Stage View showing all 4 stages.

Part D — GitHub Webhook (Auto Trigger)

In GitHub repo → Settings → Webhooks → Add webhook:

- Payload URL: <http://<EC2-Public-IP>:8080/github-webhook/>
- Content type: application/json → Just the push event → Add webhook

In Jenkins Job → Configure → Build Triggers → GitHub hook trigger for GITScm polling → Save

Now make a change to any file, commit and push — Jenkins should auto-trigger!

Expected Outcomes

✓	Create a GitHub repo and push a Jenkinsfile to it
✓	Write a multi-stage Declarative Pipeline with Build, Test, Deploy stages

✓	Use environment variables inside a Jenkinsfile
✓	Configure Jenkins Pipeline job to pull Jenkinsfile from SCM (GitHub)
✓	View the Stage View showing all pipeline stages and their durations
✓	Set up a GitHub Webhook to auto-trigger Jenkins builds on code push

Student Submission Requirements

Screenshot 1	GitHub repository showing your Jenkinsfile in the repo
Screenshot 2	Jenkins Stage View showing all 4 stages (green = success)
Screenshot 3	Full Console Output of the pipeline run
Screenshot 4	GitHub Webhook settings page showing 'Last delivery was successful'
Screenshot 5	A second build triggered automatically after a git push (build #2 or higher)

LAB 6

End-to-End CI/CD: Jenkins Deploys to EC2



Duration: 4–5 Hours



Difficulty: Advanced ★★ ★

Lab Statement

This capstone lab brings everything together. You will build a complete CI/CD pipeline where Jenkins automatically pulls code from GitHub, runs a deployment script, and deploys a web application to your EC2 instance. You will use SSH credentials in Jenkins, Jenkins environment variables, and simulate a real-world deployment workflow.

Part A — Prepare the Web Application

In your GitHub repo from Lab 5, add a simple web app:

```
mkdir -p webapp && cat > webapp/index.html << 'EOF'
<!DOCTYPE html><html>
<head><title>DevOps App</title></head>
<body>
  <h1>DevOps CI/CD Pipeline - LIVE!</h1>
  <p>Version: BUILD_VERSION_PLACEHOLDER</p>
  <p>Deployed: DEPLOY_TIME_PLACEHOLDER</p>
</body></html>
EOF
```

Part B — Create Deployment Script

```
cat > deploy.sh << 'EOF'
#!/bin/bash
set -e # Exit on any error
VERSION=$1
echo "[DEPLOY] Starting deployment of version $VERSION"
# Replace placeholders in HTML
sed -i "s/BUILD_VERSION_PLACEHOLDER/$VERSION/g" webapp/index.html
sed -i "s/DEPLOY_TIME_PLACEHOLDER/$(date)/g" webapp/index.html
# Copy to web server
sudo cp webapp/index.html /var/www/html/index.html
sudo systemctl restart httpd || sudo systemctl restart apache2
echo "[DEPLOY] Deployment v$VERSION complete!"
echo "[DEPLOY] App running at http://$(curl -s
http://169.254.169.254/latest/meta-data/public-ipv4)"
EOF
chmod +x deploy.sh
git add webapp/ deploy.sh && git commit -m 'Add webapp and deploy script'
git push
```

Part C — Update Jenkinsfile for Full CI/CD

Update your Jenkinsfile to include real deployment:

```
pipeline {
  agent any
  environment {
    VERSION = "2.0.${BUILD_NUMBER}"
    DEPLOY_SERVER = credentials('ec2-server-ip')
  }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') {
      steps {
        sh 'echo Building version ${VERSION}'
        sh 'mkdir -p artifacts'
        sh 'zip -r artifacts/app-${VERSION}.zip webapp/'
      }
    }
    stage('Test') {
      steps {
        sh 'echo Running smoke tests...'
        sh '[ -f webapp/index.html ] && echo HTML file OK'
        sh 'grep -q DevOps webapp/index.html && echo Content OK'
      }
    }
    stage('Deploy to EC2') {
      steps {
        sh 'bash deploy.sh ${VERSION}'
      }
    }
    stage('Smoke Test') {
      steps {
        sh 'curl -f http://localhost/ | grep DevOps'
        echo 'Application is live and responding!'
      }
    }
  }
  post {
    success { echo "Version ${VERSION} deployed successfully" }
    failure { echo 'CI/CD Pipeline failed. Deployment aborted.' }
  }
}
```

Part D — Run & Verify Full Pipeline

Commit and push the updated Jenkinsfile. Jenkins webhook should auto-trigger the build:

```
git add Jenkinsfile && git commit -m 'Full CI/CD pipeline v2'
git push
```

Verify the deployment by visiting <http://<EC2-Public-IP>> in your browser. You should see the live web page with the correct version number and deployment time.

BONUS

Try making a code change (e.g., change the heading text), commit & push, and verify that Jenkins auto-deploys the new version within seconds. This demonstrates the full CI/CD loop!

Expected Outcomes

✓	Create a multi-stage Declarative Pipeline that includes 5 stages: Checkout, Build, Test, Deploy, Smoke Test
✓	Use version variables (BUILD_NUMBER) in deployment filenames and web content
✓	Deploy a web application to the EC2 web server using a shell script from Jenkins
✓	Perform an automated smoke test (curl) after deployment within the pipeline
✓	Trigger the complete pipeline automatically via GitHub webhook push
✓	Verify the live application in the browser showing correct version and deploy time

Student Submission Requirements

Screenshot 1	Jenkins Stage View showing all 5 stages successful (green)
Screenshot 2	Full Console Output of the complete pipeline run
Screenshot 3	Browser showing deployed web app with version number and deploy timestamp
Screenshot 4	GitHub commit history showing at least 2 commits that triggered auto builds
Screenshot 5	Jenkins build history showing 2+ successful builds triggered by webhook
Bonus	A short video/gif or 2 screenshots showing: code change → git push → auto deploy → new version in browser